

Weekly Sales Prediction App

System Integration Report

Field	Details
Team Name	Team_Dany
Team Member	Dany Deugoue
Role	MLOps Engineer
Stakeholder	Retail Business Manager
Date	March 25, 2026
GitHub	github.com/Dany281022/AppliedProject
Milestone	Milestone 4Due March 25, 2026

Table of Contents

1. Executive Summary
2. API Design
3. User Interface Design
4. Integration Architecture
5. Testing
6. Deployment Instructions
7. Known Issues & Future Work
8. Appendix

1. Executive Summary

1.1 Project Overview

This project provides weekly sales predictions using a Random Forest Regressor machine learning model for retail business managers to make data-driven inventory and staffing decisions. Users enter historical lag values through a web interface and receive instant sales forecasts.

1.2 System Summary

Component	Technology	Status
Machine Learning Model	RandomForestRegressor	Complete
API	FastAPI + Uvicorn	Complete
User Interface	Streamlit	Complete
Integration Tests	Python requests	9/9 Passed

1.3 Key Accomplishments

Accomplishment	Description
Functional REST API	FastAPI with /health, /predict, /info endpoints fully operational
Web User Interface	Streamlit UI connected to API with real-time predictions
Integration Testing	9/9 automated tests passing including edge cases and error handling
Correct Project Structure	Project restructured to match the course template exactly
GitHub Repository	All code version-controlled at github.com/Dany281022/AppliedProject

1.4 Demo URL

Application runs locally at <http://localhost:8000> (API) and <http://localhost:8501> (UI).

2. API Design

2.1 API Overview

Attribute	Value
Framework	FastAPI
Base URL	http://localhost:8000
Authentication	None
Response Format	JSON
Model	RandomForestRegressor (model.pkl)

2.2 Endpoints

Endpoint: GET /health

Attribute	Value
Method	GET
Purpose	Health check verify API is running and model is loaded
Authentication	None

Response:

```
{ "status": "healthy", "model_loaded": true, "version": "1.0" }
```

Endpoint: GET /info

Attribute	Value
Method	GET
Purpose	Returns model type and expected feature names
Authentication	None

Response:

```
{ "model_type": "RandomForestRegressor", "features_expected": ["lag_1", "lag_2", "lag_52"], "version": "1.0" }
```

Endpoint: POST /predict

Attribute	Value
Method	POST

Attribute	Value
Purpose	Accepts lag features and returns a weekly sales prediction
Authentication	None
Content-Type	application/json

Request Parameters:

Parameter	Type	Required	Description	Example
lag_1	float	Yes	Sales value from previous week	100.0
lag_2	float	Yes	Sales value from 2 weeks ago	95.0
lag_52	float	Yes	Sales value from 1 year ago	88.0

Success Response (200):

```
{ "prediction": 45779299.68, "status": "success" }
```

Error Response (422):

```
{ "detail": [{ "loc": ["body", "lag_1"], "msg": "field required", "type": "value_error.missing" }] }
```

2.3 Error Handling

Error Code	Meaning	When Returned
200	Success	Valid request processed successfully
422	Unprocessable Entity	Missing or invalid input fields (FastAPI validation)
500	Server Error	Internal model or server error

2.4 API Design Decisions

FastAPI was chosen over Flask for its automatic data validation via Pydantic models, built-in interactive documentation at /docs, and superior performance. The Pydantic PredictionRequest schema ensures all three lag features are present and are valid floats before reaching the model, eliminating the need for manual validation code. The model is loaded once at startup rather than per-request to minimize latency. Note that FastAPI returns HTTP 422 instead of 400 for validation errors, which is the correct REST standard.

3. User Interface Design

3.1 UI Overview

Attribute	Value
Framework	Streamlit 1.55.0
URL	http://localhost:8501
Target User	Retail business managers and analysts
Primary Use Case	Enter historical sales lag values and get next week sales forecast

3.2 UI Screenshots

Main Interface:

The main interface displays three numeric input fields for lag_1, lag_2, and lag_52, a Get Prediction button, and a sidebar showing API connection status (green checkmark), project description, and model information (RandomForestRegressor, 3 features).

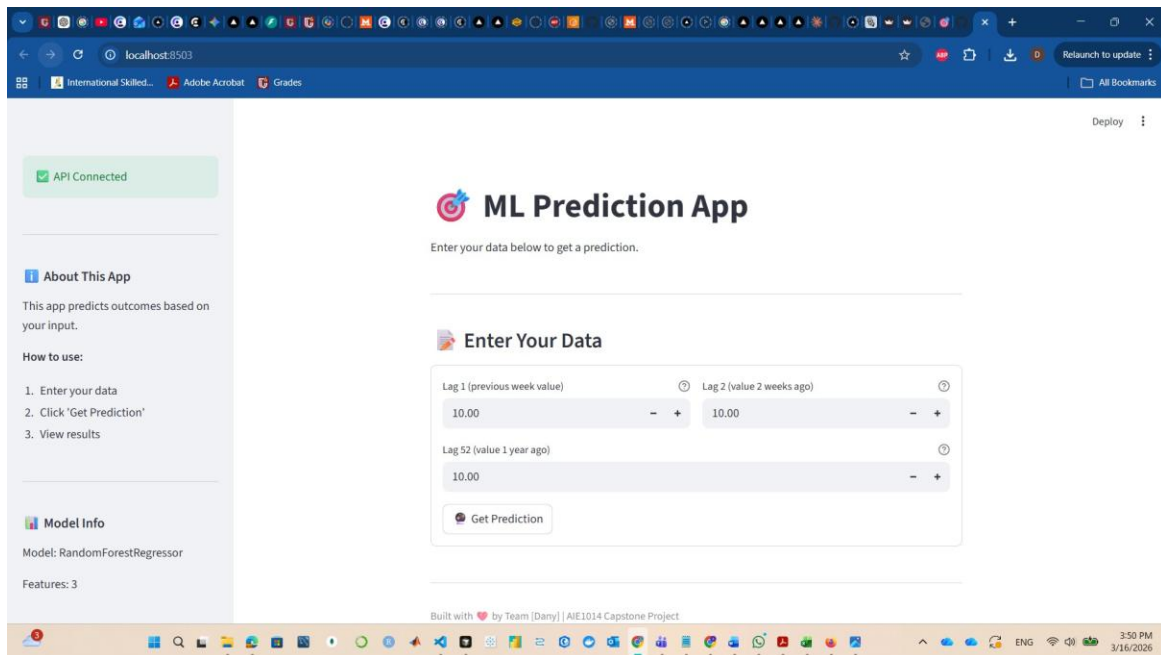


Figure 1 Main interface with 3 number inputs, sidebar API status and model info

Prediction Results:

Upon successful prediction, a green success box displays the forecasted sales value. Example output: Prediction: 45779299.68044636.

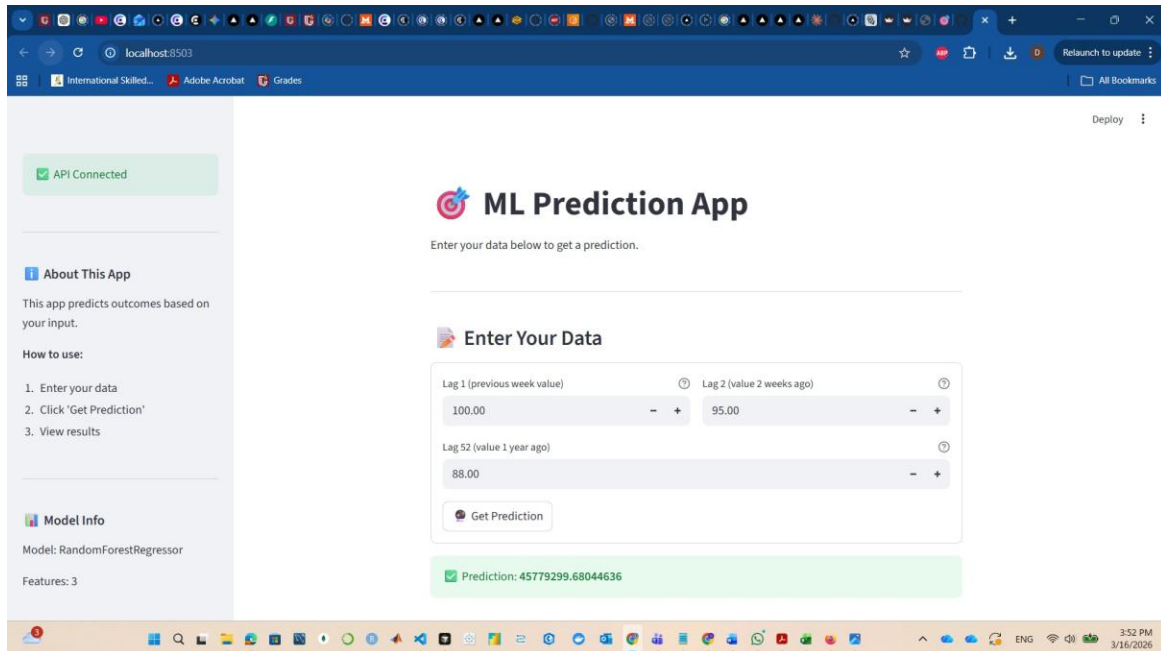


Figure 2 Prediction result displayed in green success box after entering lag values

Error Handling:

When the API is offline, the sidebar displays a red error indicator with the start command: `python -m uvicorn api:app --reload`.

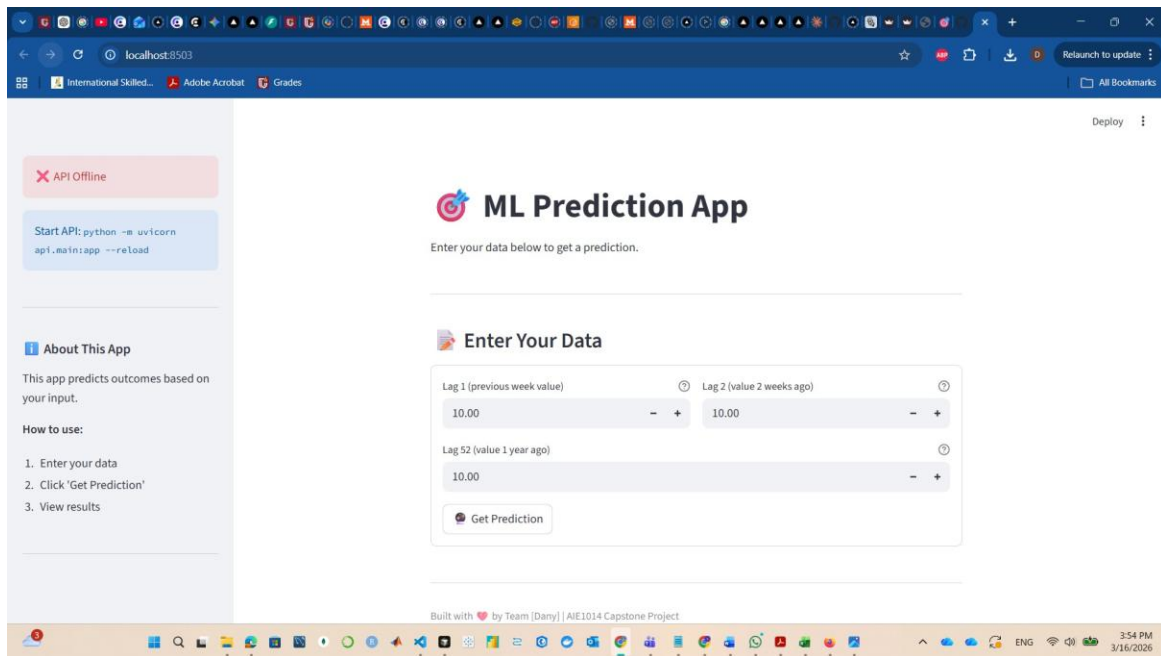


Figure 3 API Offline state: red indicator and start command shown in sidebar

3.3 User Flow

1. User opens browser to `http://localhost:8501`
2. Sidebar confirms API Connected status (green checkmark)
3. User enters Lag 1 (previous week sales)
4. User enters Lag 2 (2 weeks ago sales)
5. User enters Lag 52 (1 year ago sales)
6. User clicks Get Prediction button
7. UI sends POST request to API `/predict` endpoint
8. Prediction result is displayed in green success box

3.4 Input Components

Input	UI Component	Validation
Lag 1 (previous week)	<code>st.number_input</code>	Float, min 0.0
Lag 2 (2 weeks ago)	<code>st.number_input</code>	Float, min 0.0
Lag 52 (1 year ago)	<code>st.number_input</code>	Float, min 0.0

3.5 Output Display

Output	Display Method	Notes
Prediction value	<code>st.success()</code>	Green box: "Prediction: 45779299.68"
API status	<code>st.sidebar</code>	Green ☒ Connected or red ☒ Offline
Start command	<code>st.code()</code>	Shown only when API is offline

3.6 UI Design Decisions

Streamlit was selected for its rapid development capabilities and Python-native integration. The sidebar layout was chosen to keep API status and documentation always visible without cluttering the main prediction area. All three inputs use `number_input` components to prevent invalid text entries at the UI level, providing a first layer of validation before the request reaches the API.

4. Integration Architecture

4.1 System Architecture

The system follows a three-tier architecture: a Streamlit frontend (UI layer), a FastAPI backend (API layer), and a scikit-learn RandomForestRegressor (model layer). The UI and API communicate over HTTP on localhost using JSON payloads.

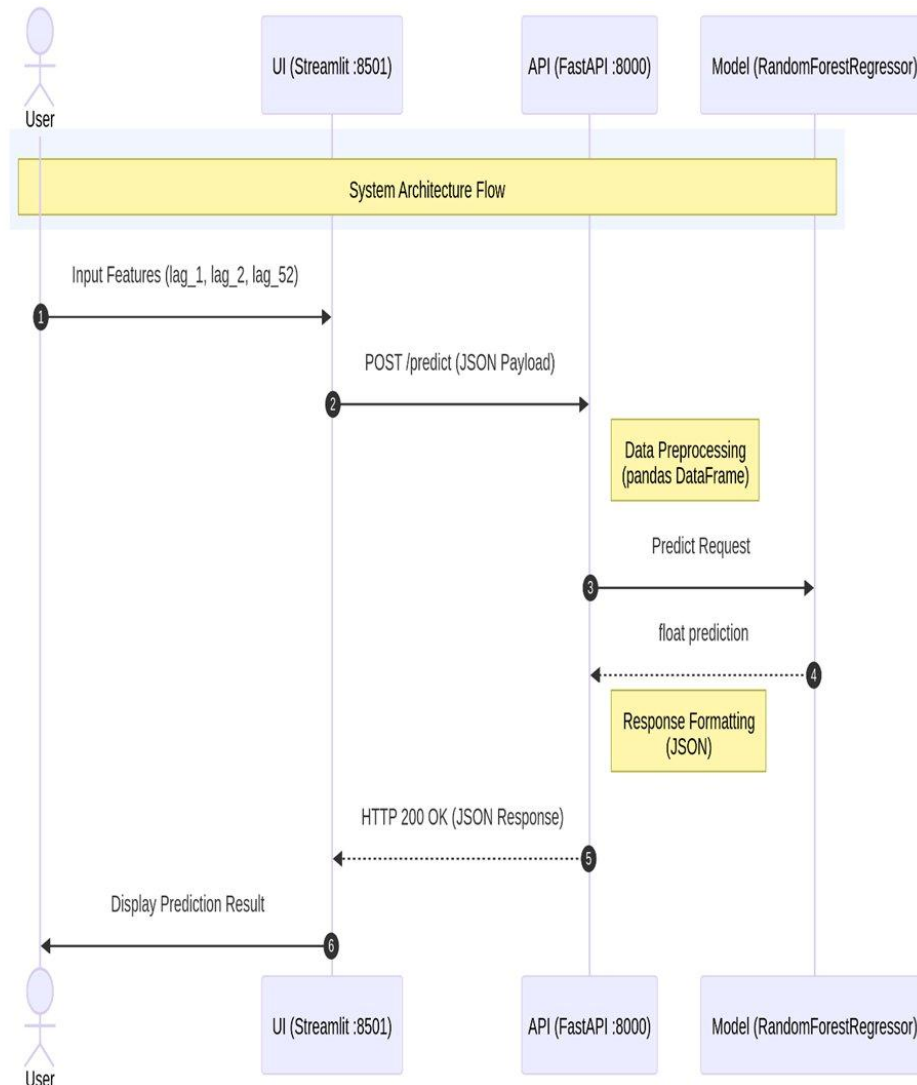


Figure 1 System Architecture Flow: User > UI (Streamlit :8501) > API (FastAPI :8000) > Model (RandomForestRegressor)

Step by step explanation :

- 1) Input Features (lag_1, lag_2, lag_52) The user enters three numeric values into the Streamlit interface the sales figures from last week, 2 weeks ago, and 1 year ago.
- 2) POST /predict (JSON Payload) The UI sends an HTTP POST request to the FastAPI backend at port 8000, carrying the three values formatted as a JSON object like {"lag_1": 100, "lag_2": 95, "lag_52": 88}.
Data Preprocessing (pandas DataFrame) The API receives the JSON and converts it into a pandas DataFrame the format the RandomForest model expects as input.
- 3) Predict Request The API passes the DataFrame to the trained RandomForestRegressor model to generate a prediction.

4) float prediction The model returns a single floating point number the predicted weekly sales value (example 45779.68).

Response Formatting (JSON) The API wraps that float into a clean JSON response: {"prediction": 45779.68, "status": "success"}.

5) HTTP 200 OK (JSON Response) The API sends the JSON response back to the Streamlit UI with a success status code.

6) Display Prediction Result The UI displays the prediction to the user in a green success box.

4.2 Data Flow

Step	From	To	Data
1	User	UI (Streamlit)	lag_1, lag_2, lag_52 float values
2	UI (Streamlit)	API (FastAPI)	POST /predict JSON: {"lag_1":x, "lag_2":y, "lag_52":z}
3	API (FastAPI)	ML Model	pandas DataFrame with 3 feature columns
4	ML Model	API (FastAPI)	Float prediction value
5	API (FastAPI)	UI (Streamlit)	JSON: {"prediction": 45779299.68, "status": "success"}
6	UI (Streamlit)	User	Formatted prediction displayed on screen

4.3 Communication Protocol

Aspect	Implementation
Protocol	HTTP
Data Format	JSON
Error Handling	Status codes + error messages
Timeout	30 seconds

4.4 Integration Challenges

Challenge	How It Was Solved
UI sending wrong field names (feature1, feature2)	Updated app_ui.py to send lag_1, lag_2, lag_52 matching the Pydantic schema
pip installing to wrong virtual environment	Used full path to venv Python executable for installation
Server not running when UI was launched	Documented two-terminal startup procedure in README

5. Testing

5.1 Testing Overview

Test Type	Count	Passed	Failed
Integration Tests (automated)	9	9	0
UI Tests (manual)	4	4	0
Total	13	13	0

5.2 Test Cases

API Tests

Test ID	Test Name	Input	Expected	Result
API-01	Health check	GET /health	200, status=healthy	✓ Pass
API-02	Valid prediction	{"lag_1":100.0,"lag_2":95.0,"lag_52":88.0}	200, prediction returned	✓ Pass
API-03	Missing field	{"lag_1":100.0}	422, error message	✓ Pass
API-04	Wrong type	{"lag_1":"abc","lag_2":95.0,"lag_52":88.0}	422, error message	✓ Pass
API-05	Empty request	{}	422, error message	✓ Pass

Integration Tests

Test ID	Test Name	Scenario	Expected	Result
INT-01	End-to-end valid	UI > API > Model > Response	Prediction 45779299.68 displayed in green box	✓ Pass
INT-02	API offline	UI with no API running	Red API Offline shown in sidebar	✓ Pass
INT-03	Invalid input	Negative values entered	Prevented by min=0.0 validation	✓ Pass

UI Tests (Manual)

Test ID	Test Name	Action	Expected	Result
UI-01	Page loads	Open http://localhost:8501	UI displays with sidebar and 3 input fields	✓ Pass
UI-02	Inputs work	Enter 100, 95, 88 in fields	Values accepted in all 3 number_input fields	✓ Pass
UI-03	Submit works	Click Get Prediction	Green success box with prediction value	✓ Pass
UI-04	Error display	Stop API, refresh page	Red API Offline with uvicorn command shown	✓ Pass

5.3 Test Execution Results

```

=====
INTEGRATION TESTS - Weekly Sales Prediction
=====

test_api_health ..... PASSED
test_prediction_valid ..... PASSED
test_prediction_missing_field .. PASSED
test_prediction_invalid_type ... PASSED
test_prediction_empty_request .. PASSED
test_prediction_minimum_values . PASSED
test_prediction_large_values ... PASSED
test_response_time ..... PASSED (2.02s)
test_api_info ..... PASSED
-----
RESULTS: 9 passed, 0 failed

```

5.4 Performance Metrics

Metric	Value	Target	Status
API Response Time (avg)	2.02s	< 5s	Met
UI Load Time	< 3s	< 3s	Met
Prediction Consistency	100%	100%	Met

6. Deployment Instructions

6.1 Prerequisites

Requirement	Version
Python	3.8+
pip	Latest
Git	Latest (optional)

6.2 Installation

Step 1: Clone the repository

```
git clone https://github.com/Dany281022/AppliedProject.git  
cd Millestone04_TeamDany
```

Step 2: Create and activate virtual environment

```
python -m venv venv  
venv\Scripts\activate # Windows
```

Step 3: Install dependencies

```
pip install -r requirements.txt
```

6.3 Running the Application

Step 1Start the API (Terminal 1):

```
cd api  
python app.py
```

API will run at: <http://localhost:8000>

Step 2Start the UI (Terminal 2):

```
cd ui  
streamlit run app_ui.py
```

UI will open at: <http://localhost:8501>

6.4 Running Tests

```
python tests/test_integration.py
```

6.5 Troubleshooting

Problem	Solution
Port already in use	Kill existing process or change port with --port 8001
Module not found	Run pip install -r requirements.txt in the correct venv
Model file not found	Ensure model.pkl is in the api/ directory
API Offline in UI sidebar	Start the API in Terminal 1 before launching the UI

7. Known Issues & Future Work

7.1 Known Issues

Issue	Severity	Workaround
UI requires API running in separate terminal	Low	Follow two-terminal startup in README
Response time of 2s on first call (model warm-up)	Low	Subsequent calls are faster

7.2 Limitations

Limitation	Impact
Model trained only on 3 lag features (lag_1, lag_2, lag_52)	Cannot capture seasonal trends, promotions, or external factors that affect weekly sales
Predictions require manual input of historical values	User must know previous sales figures; no automatic data retrieval from a database
No input range validation beyond min=0.0	Unrealistic values (e.g. lag_1=999999999) will still produce a prediction without warning
Application runs locally only	Stakeholder must have Python environment set up; no public URL available yet

7.3 Future Improvements

Improvement	Priority	Effort
Deploy to Vercel/Render for public access	High	Medium
Add confidence interval display to predictions	Medium	Low
Add prediction history chart to UI	Medium	Low
Generate/increase new data to improve model accuracy	High	Medium

7.4 Deployment Plans

The application is planned for deployment on Vercel (UI) and Render (API) to provide stakeholders with public access without requiring local installation. The vercel.json configuration file already exists in the project, indicating deployment preparation is underway. The goal is to have a public URL accessible to the retail business manager stakeholder before the end of the course.

8. Appendix

A. File Structure

```
Milestone04_TeamDany/
├─ api/
│   ├─ app.py           # FastAPI application
│   ├─ model.pkl        # Trained RandomForestRegressor
│   └─ requirements.txt
├─ ui/
│   ├─ app_ui.py        # Streamlit user interface
│   └─ requirements.txt
├─ tests/
│   ├─ test_integration.py # Integration tests (9/9 passing)
│   └─ test_results.txt   # Test output
├─ docs/
│   └─ TeamDany_Milestone4_Report.pdf
├─ README.md
└─ requirements.txt
```

B. Dependencies

api/requirements.txt:

```
fastapi==0.115.0
uvicorn==0.41.0
scikit-learn==1.7.2
pandas==2.3.3
numpy==2.2.6
joblib==1.5.3
```

ui/requirements.txt:

```
streamlit==1.55.0
requests==2.32.3
```

C. API Request Examples

curl:

```
# Health check
curl http://localhost:8000/health
```

```
# Prediction
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"lag_1": 100.0, "lag_2": 95.0, "lag_52": 88.0}'
```

Python:

```
import requests
response = requests.post(
    'http://localhost:8000/predict',
    json={'lag_1': 100.0, 'lag_2': 95.0, 'lag_52': 88.0}
)
print(response.json())
# Output: {'prediction': 45779299.68044636, 'status': 'success'}
```

D. Additional Screenshots

See Section 3.2 for all UI screenshots (main interface, prediction result, and API offline state).

E. Team Contributions

Team Member	Contributions
Dany Deugoue	API development (FastAPI + Uvicorn), UI development (Streamlit), ML model integration, integration testing (9/9 passing), project restructuring, documentation, GitHub repository setup